

Florence: Infrastructure - OpenRouter

1. Overview and Purpose

OpenRouter serves as the unified API gateway and routing layer for all Large Language Model interactions within the Florence platform. Instead of hardcoding specific AI provider APIs the backend microservices utilise OpenRouter to dynamically route requests to various foundation models. This ensures high availability, automatic failover and simplified billing.

2. Configuration and Secrets Management

Environment variables are required by the Python microservices to authenticate and route AI requests.

Environment Variable	Description	Where it is stored	Required By
LLM_API_KEY	The private OpenRouter API key used to authenticate requests.	Vercel Environment Variables	LLM Engine Service, LLM Chatbot Service
LLM_BASE_URL	The OpenRouter API endpoint.	Vercel Environment Variables	LLM Engine Service, LLM Chatbot Service
LLM_MODEL	The specific model identifier string passed to the API.	Vercel Environment Variables	LLM Engine Service, LLM Chatbot Service

3. Technical Implementation Details

The backend utilises a centralised `LLMFactory` class within the LLM Engine Service to instantiate LangChain chat models. This factory reads the environment variables and

configures the model parameters such as temperature and max tokens.

Current Models Utilised

- Gemini 3.1 Pro Preview:** Used to be the primary model for complex reasoning, risk assessment and detailed clinical insights. This model used to account for the majority of the usage.
- Gemini 3 Flash Preview:** Used to be utilised for faster and lower latency tasks such as basic chatbot responses and simple data extraction.
- Gemini 3.5 Flash:** Currently configured as the sole and main model for the platform.

4. Billing, Limits and Day 2 Operations

- Pricing Model:** Pay per token. Costs are incurred based on the input and output tokens processed by the selected models.
- Limits to Watch:** There are currently no budget limits or credit limits configured on the active API key. Credits must be monitored at the dashboard in OpenRouter to ensure the account balance is maintained.
- Production Safeguards:** To prevent unexpected costs in a live production environment it is highly recommended to configure a hard Credit Limit or set up Budget alerts within the OpenRouter dashboard before scaling the user base.
- Day 2 Operations:** To switch to a different AI provider or model, the development team simply updates the `LLM_MODEL` environment variable in Vercel without changing any backend code.